

SECURITY RESEARCH SERIES

INFRASTRUCTURA & CACHÉ

CORS // POISONING // DECEPTION

AGENDA TÉCNICA DE LA SESIÓN

01. CORS Exploit

Anatomía de la política de origen, manipulación de encabezados y robo de PII mediante reflexión y orígenes nulos.

02. Web Cache Poisoning

Envenenamiento de CDNs mediante entradas no trackeadas (unkeyed inputs) y escalado a XSS masivo.

03. Web Cache Deception

Exfiltración de datos privados aprovechando discrepancias en el mapeo de rutas entre Proxies y Backends.

BLOQUE 01

CORS SECURITY

SOP: SAME-ORIGIN POLICY

El SOP es el pilar de seguridad de los navegadores modernos. Impide que un script de sitio-a.com lea datos de sitio-b.com.

Definición de Origen:

- Protocolo (HTTPS)
- Host (qbit.io)
- Puerto (443)



CORS: RELAJACIÓN DEL SOP

CORS permite a los servidores definir quién puede saltarse el SOP. El servidor declara orígenes de confianza mediante cabeceras HTTP específicas.

```
GET /api/data HTTP/1.1
Origin: https://malicious-site.com

HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://malicious-site.com
Access-Control-Allow-Credentials: true
```

Si el servidor refleja el Origin, la seguridad está comprometida.

ANÁLISIS DE CABECERAS CORS

Cabecera	Descripción Técnica
Origin	Enviada por el navegador para indicar de dónde viene el script.
AC-Allow-Origin	Indica qué dominio tiene permiso de lectura.
AC-Allow-Credentials	Si es 'true', permite enviar Cookies/Auth en peticiones cross-origin.
AC-Expose-Headers	Permite al JS leer cabeceras de respuesta personalizadas.

VECTOR: REFLEXIÓN DE ORIGEN

Ocurre cuando el servidor genera la respuesta Access-Control-Allow-Origin dinámicamente basándose únicamente en el valor de la cabecera Origin de la petición.

```
// Atacante envía:  
Origin: https://attacker.com  
  
// Servidor responde:  
Access-Control-Allow-Origin: https://attacker.com  
Access-Control-Allow-Credentials: true
```

Cualquier sitio web puede ahora robar la sesión de la víctima mediante JS.

VECTOR: TRUSTED NULL ORIGIN

El origen null se genera en redirecciones, archivos locales o iframes sandboxed. Algunos desarrolladores lo incluyen en su lista de confianza por error.

```
</div>
```

Explotación

El atacante usa un iframe con sandbox para forzar que el navegador envíe Origin: null, puenteando filtros de dominio inadecuados.

LAB: BASIC ORIGIN REFLECTION

Análisis del Escenario

El servidor es perezoso y refleja cualquier origen. El objetivo es alojar un script en el servidor de exploits que realice un fetch a /accountDetails con credenciales.

Resultado: El atacante exfiltra la API Key del usuario administrador cuando este visita la página maliciosa.

MITIGACIÓN ESTRATÉGICA: CORS

Defensa

- Usar una **Whitelist estática** de dominios permitidos.
- No permitir credenciales si el origen es *.
- Validar estrictamente el origen (evitar Regex laxos).



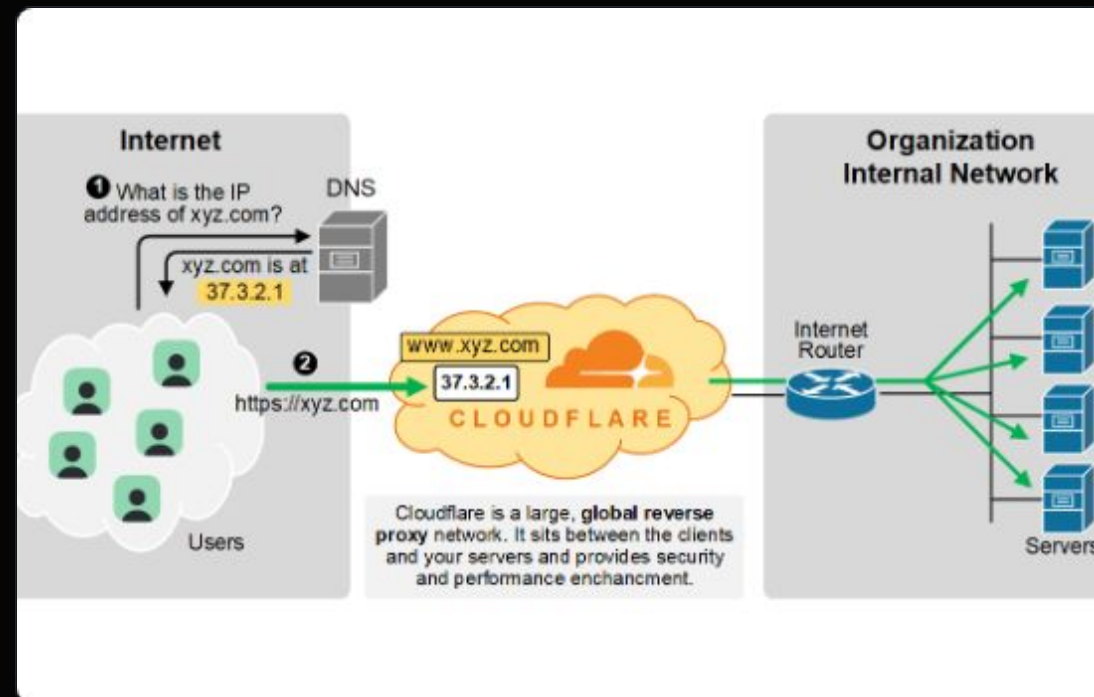
BLOQUE 02

CACHE POISONING

ARQUITECTURA DE CACHE WEB

La caché vive entre el servidor y el usuario (CDNs como Cloudflare o Akamai). Su función es reducir latencia sirviendo copias de páginas populares.

```
X-Cache: hit  
Age: 154
```



CACHE KEYS VS UNKEYED INPUTS

La caché decide si sirve una copia basándose en la **Cache Key** (usualmente URL y cabecera Host).

Cache Key

Lo que la caché mira para identificar el recurso.

Unkeyed Input

Cabeceras (ej. X-Forwarded-Host) que el servidor usa para generar la página pero que la caché ignora.

CICLO DEL ENVENENAMIENTO

1. Atacante envía **Unkeyed Input** malicioso → 2. Servidor genera respuesta infectada → 3. Caché guarda el veneno → 4. Víctimas reciben el veneno.

```
GET /home HTTP/1.1
X-Forwarded-Host: evil.com

HTTP/1.1 200 OK
X-Cache: miss (Ahora guardado)
... </div>
```

VECTOR: X-FORWARDED-HOST

Utilizado por aplicaciones para construir URLs dinámicas en el HTML. Si el atacante inyecta un host externo y la caché lo guarda, el ataque se vuelve masivo.

Impacto

El atacante controla los recursos estáticos (JS/CSS) de la página principal para todos los usuarios concurrentes.

VECTOR: UNKEYED COOKIE

Ocurre cuando el servidor usa un valor de una cookie para renderizar parte del DOM pero la caché no incluye esa cookie en su clave de búsqueda.

```
Cookie: language=es"; alert(1); //
```

La respuesta con el alert() se queda grabada en la caché del servidor y se sirve a usuarios que ni siquiera tienen esa cookie.

TÉCNICA: CACHE BUSTER

Durante la auditoría, usamos un parámetro aleatorio en la URL para no afectar a otros usuarios y forzar una actualización de la caché.

```
GET /home?cb=12345
```

Propósito

Permite probar inyecciones repetidamente sin esperar a que el TTL (Time To Live) de la caché original expire.

HERRAMIENTA: PARAM MINER

Extensión de Burp Suite diseñada para descubrir "Unkeyed Inputs" de forma automática mediante fuerza bruta de cabeceras y parámetros.

- Adivina cabeceras HTTP ocultas.
- Detecta discrepancias de caché.
- Esencial para Poisoning avanzado.



MITIGACIÓN: CACHE MANAGEMENT

- Solo cachear respuestas que tengan cabeceras Cache-Control explícitas.
- Incluir cabeceras que afecten la respuesta en la **Cache Key**.
- Usar la cabecera Vary para indicar qué inputs afectan el contenido.
- Deshabilitar cabeceras X-Forwarded-* si no son necesarias.

BLOQUE 03

CACHE DECEPTION

DECEPTION VS POISONING

Atributo	Cache Poisoning	Cache Deception
Dirección	Atacante → Caché → Víctimas	Víctima → Caché → Atacante
Objetivo	XSS Masivo / Defacement	Exfiltración de PII / Datos Admin
Mecanismo	Unkeyed Input	Path Discrepancy (Rutas)

MECÁNICA DEL ENGAÑO

Aprovechamos la confusión de rutas. El Proxy cree que estamos pidiendo una imagen estática, pero el Backend devuelve datos privados.

```
GET /my-account/profile.jpg
```

El Error

La caché guarda el perfil del usuario pensando que es una imagen inofensiva y pública.

DISCREPANCIA EN PATH MAPPING

Servidores web diferentes interpretan el final de la URL de forma distinta:

- **Nginx/Proxy:** "Termina en .jpg, lo guardo en caché".
- **Backend (Rails/Django):** "Ignoro .jpg, el recurso es /profile".

El atacante atrae a la víctima a la URL fake y luego la visita él mismo para obtener la copia cacheada.

VECTOR: DELIMITADORES CONFUSOS

Uso de caracteres que el proxy ignora pero el backend usa para mapear:

- Punto y coma: `/profile;test.js`
- Signo de interrogación: `/profile?test.jpg`
- Hash: `/profile#test.css`

Si el proxy cachea todo lo que termina en `.css`, el perfil privado queda expuesto bajo esa ruta falsa.

IMPACTO: EXFILTRACIÓN DE PII

A diferencia de CORS, este ataque es **Zero-Click** para el atacante tras el engaño inicial. No necesita scripts maliciosos complejos.

Datos Robados:

- Emails / Direcciones
- Tokens CSRF
- Claves de API en UI



LAB: PATH MAPPING DECEPTION

Lógica de Resolución

1. Engañar al administrador para que visite `/my-account/profile/test.js`.
2. El proxy cachea la respuesta porque termina en `.js`.
3. El atacante entra a esa URL y **visualiza la API Key del administrador** descargada de la caché.

MITIGACIÓN: CACHE DECEPTION

Protección

- Usar Cache-Control: no-store para datos sensibles.
- Configurar cabecera X-Content-Type-Options: nosniff.
- Asegurar que el proxy y el backend usen la misma lógica de mapeo.



RESUMEN DE SUPERFICIE DE ATAQUE

Ataque	Causa	Solución
CORS	Confianza en Origin	Whitelist Estática
Poisoning	Unkeyed Inputs	Cache Key Hardening
Deception	Path Confusion	Cache-Control Headers

CHECKLIST DEL ESPECIALISTA

Auditoría CORS

- ¿Refleja Origin?
- ¿Acepta Null?

Auditoría Caché

- ¿Headers unkeyed?
- ¿Confusión de .js/.jpg?



¿PREGUNTAS?